Project - Quran Ayah Correction Chatbot (LAB) 👨‍💻📖

1️⃣ Required Libraries

In [4]:
```
# Basic Libraries
import os
import re
import gc
from collections import Counter


import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# NLP / Retrieval Libraries
from rapidfuzz import fuzz
from sentence_transformers import SentenceTransformer

# Audio Display
from IPython.display import Audio, display

pd.set_option("display.max_colwidth", 160)
plt.style.use("seaborn-v0_8")
```

2️⃣ Read Dataset

Read From GitHub

In [5]:
```
DATA_URL = "https://raw.githubusercontent.com/malekverse/quran-dataset/main/quran_d
```

In [6]:
```
df_load = pd.read_csv(DATA_URL)
```

📁 Project Data Source: Quran Dataset

1. Overview

This project uses a complete Quran dataset. The dataset contains the Arabic Quran text and metadata for each ayah.

2. Important Columns

Column	Description
`surah_no`	Surah number
`surah_name_ar`	Arabic surah name
`ayah_no_surah`	Ayah number inside the surah
`ayah_no_quran`	Global ayah number in the Quran
`ayah_ar`	Arabic ayah text
`ayah_en`	English translation

3. Project Goal

The goal is to build a chatbot that corrects Quran ayah input by retrieving the closest correct ayah from the dataset.

The model is used for retrieval only. It does not generate Quranic text.

Split to Columns

In [7]:
```
df = df_load.rename(
    columns={
        "surah_name_ar": "surah_name",
        "ayah_no_surah": "ayah_number",
        "ayah_ar": "text",
    }
)

df = df[
    [
        "surah_no",
        "surah_name",
        "ayah_number",
        "ayah_no_quran",
        "text",
        "ayah_en",
    ]
].copy()
```

In [8]:
```
# Keep a local copy for the Streamlit chatbot app
os.makedirs("data", exist_ok=True)
df.to_csv("data/quran_raw.csv", index=False, encoding="utf-8")
print("Data saved successfully to your disk!")
```

```
Data saved successfully to your disk!
```

In [9]:
```
# You can also load it locally after saving:
# df = pd.read_csv("data/quran_raw.csv")
# print(f"Loaded {len(df)} rows. Ready for processing.")
```

In [10]:
```
df.shape
```

Out[10]: (12472, 6)

In [11]:
```
df.head()
```

Out[11]:

	surah_no	surah_name	ayah_number	ayah_no_quran	text	ayah_en
0	1	الفاتحة	1	1	بِسْمِ ٱللَّهِ ٱلرَّحْمَٰنِ ٱلرَّحِيمِ	In the Name of Allah —the Most Compassionate, Most Merciful.
1	1	الفاتحة	2	2	ٱلْحَمْدُ لِلَّهِ رَبِّ ٱلْعَٰلَمِينَ	All praise is for Allah —Lord of all worlds,
2	1	الفاتحة	3	3	ٱلرَّحْمَٰنِ ٱلرَّحِيمِ	the Most Compassionate, Most Merciful,
3	1	الفاتحة	4	4	مَٰلِكِ يَوْمِ ٱلدِّينِ	Master of the Day of Judgment.
4	1	الفاتحة	5	5	إِيَّاكَ نَعْبُدُ وَإِيَّاكَ نَسْتَعِينُ	You ˹alone˺ we worship and You ˹alone˺ we ask for help.

In [12]:
```
if "df_load" in locals():
    del df_load
gc.collect()
```

Out[12]: 80

3️⃣ Exploratory Data Analysis (EDA)

Information

In [13]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12472 entries, 0 to 12471
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype 
---  ------         --------------  ----- 
 0   surah_no       12472 non-null  int64 
 1   surah_name     12472 non-null  object
 2   ayah_number    12472 non-null  int64 
 3   ayah_no_quran  12472 non-null  int64 
 4   text           12472 non-null  object
 5   ayah_en        12472 non-null  object
dtypes: int64(3), object(3)
memory usage: 584.8+ KB
```

Check Columns

In [14]:
```
df.columns
```

Out[14]:
```
Index(['surah_no', 'surah_name', 'ayah_number', 'ayah_no_quran', 'text',
       'ayah_en'],
      dtype='object')
```

Description

In [15]:
```
df.describe(include='all')
```

Out[15]:

	surah_no	surah_name	ayah_number	ayah_no_quran	text	ayah_en
count	12472.000000	12472	12472.000000	12472.000000	12472	12472
unique	NaN	114	NaN	NaN	6065	6092
top	NaN	البقرة	NaN	NaN	فَبِأَيِّ ءَالَآءِ رَبِّكُمَا تُكَذِّبَانِ	Then which of your Lord's favours will you both deny?
freq	NaN	572	NaN	NaN	62	60
mean	33.519724	NaN	53.506575	3118.500000	NaN	NaN
std	26.460200	NaN	50.461901	1800.250289	NaN	NaN
min	1.000000	NaN	1.000000	1.000000	NaN	NaN
25%	11.000000	NaN	16.000000	1559.750000	NaN	NaN
50%	26.000000	NaN	38.000000	3118.500000	NaN	NaN
75%	51.000000	NaN	75.000000	4677.250000	NaN	NaN
max	114.000000	NaN	286.000000	6236.000000	NaN	NaN

Check Duplications

In [16]:
```
print("Duplicated rows:", df.duplicated().sum())
print("Duplicated global ayah numbers:", df.duplicated("ayah_no_quran").sum())
```

```
Duplicated rows: 6236
Duplicated global ayah numbers: 6236
```

In [17]:
```
df.drop_duplicates(subset=["ayah_no_quran"], keep="first", inplace=True)
df.reset_index(drop=True, inplace=True)
df.shape
```

Out[17]:
```
(6236, 6)
```

Check Missing Values

In [18]:
```
df.isna().sum()
```

Out[18]:
```
surah_no         0
surah_name       0
ayah_number      0
ayah_no_quran    0
text             0
ayah_en          0
dtype: int64
```

Show Number of Unique Values

In [19]:
```
for col in df.columns:
    print("{} : {} unique value(s)".format(col, df[col].nunique()))
```

```
surah_no : 114 unique value(s)
surah_name : 114 unique value(s)
ayah_number : 286 unique value(s)
ayah_no_quran : 6236 unique value(s)
text : 6065 unique value(s)
ayah_en : 6092 unique value(s)
```

Show Most Commen in Columns

In [20]:
```
top_surahs = df["surah_name"].value_counts().head(15).reset_index()
top_surahs.columns = ["Surah", "Number of Ayahs"]
top_surahs
```

Out[20]:

	Surah	Number of Ayahs
0	البقرة	286
1	الشعراء	227
2	الأعراف	206
3	آل عمران	200
4	الصافات	182
5	النساء	176
6	الأنعام	165
7	طه	135
8	التوبة	129
9	النحل	128
10	هود	123
11	المائدة	120
12	المؤمنون	118
13	الأنبياء	112
14	يوسف	111

In [21]:
```
all_words = " ".join(df["text"].astype(str)).split()
word_counts = Counter(all_words)

top_words = pd.DataFrame(word_counts.most_common(15), columns=["Word", "Frequency"]
top_words
```

Out[21]:

	Word	Frequency
0	ۚ	1972
1	ۖ	1682
2	فِى	1098
3	ٱللَّهِ	830
4	ٱلَّذِينَ	810
5	ٱللَّهُ	733
6	مِن	728
7	مَا	711
8	لَا	616
9	إِنَّ	609
10	وَلَا	605
11	ۗ	603
12	ٱللَّهَ	592
13	إِلَّا	562
14	وَمَا	512

Make Columns Length of Sentence

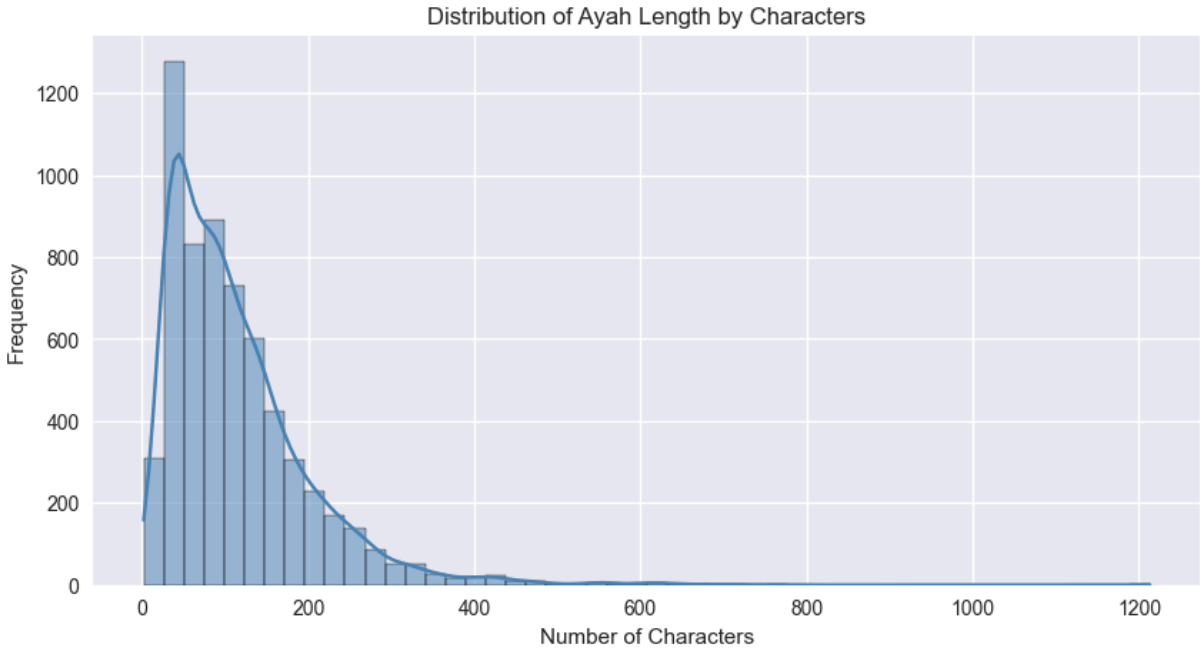
In [22]:
```
df["number_of_characters_ayah"] = df["text"].astype(str).str.len()
df["number_of_words_ayah"] = df["text"].astype(str).str.split().str.len()
```

In [23]:
```
print(f"Maximum number of characters in an ayah is: {df['number_of_characters_ayah'
print(f"Minimum number of characters in an ayah is: {df['number_of_characters_ayah'
print(f"Average number of characters in an ayah is: {df['number_of_characters_ayah'
```

```
Maximum number of characters in an ayah is: 1213
Minimum number of characters in an ayah is: 2
Average number of characters in an ayah is: 113.97
```

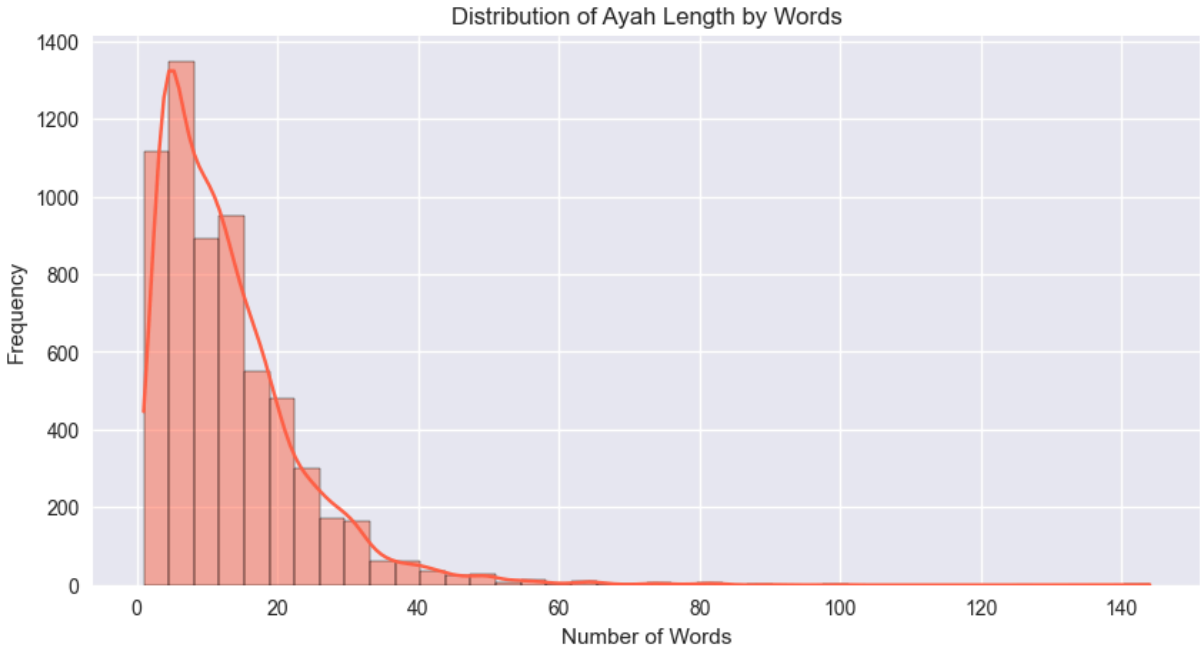
Distribution of Length Characters

In [24]:
```
plt.figure(figsize=(10, 5))
sns.histplot(df["number_of_characters_ayah"], bins=50, kde=True, color="steelblue")
plt.title("Distribution of Ayah Length by Characters")
plt.xlabel("Number of Characters")
plt.ylabel("Frequency")
plt.show()
```



Distribution of Length Words

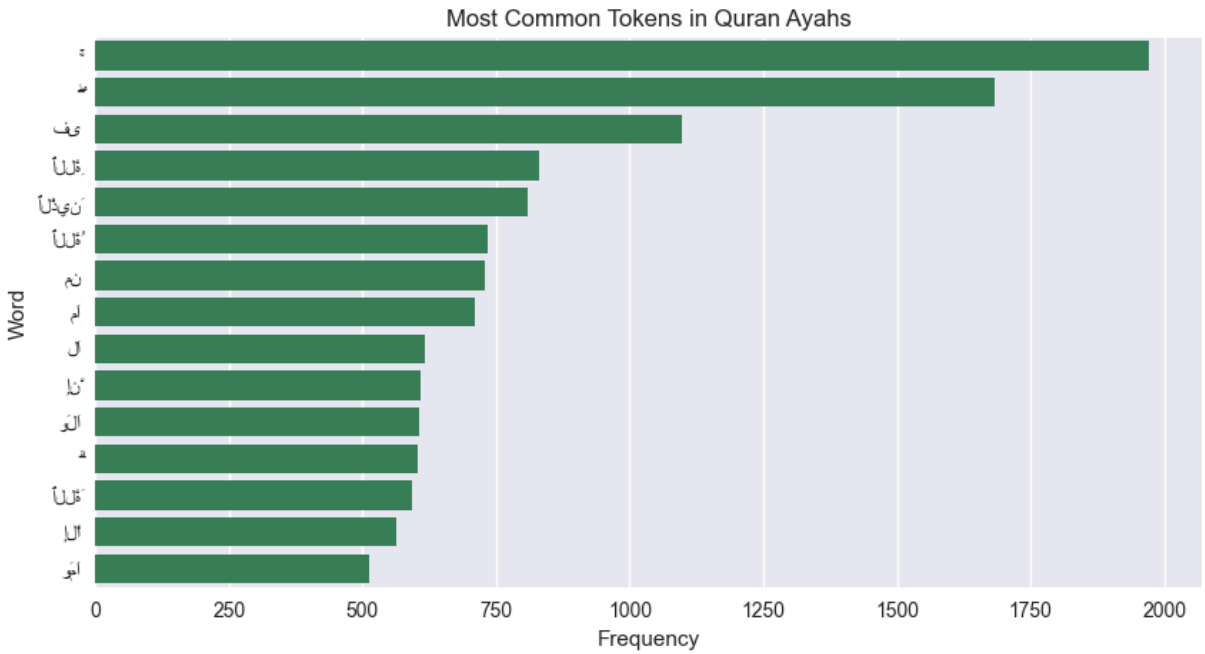
In [25]:
```
plt.figure(figsize=(10, 5))
sns.histplot(df["number_of_words_ayah"], bins=40, kde=True, color="tomato")
plt.title("Distribution of Ayah Length by Words")
plt.xlabel("Number of Words")
plt.ylabel("Frequency")
plt.show()
```



Generating Word Cloud For Sentences

In [26]:
```
# A bar chart is used instead of a word cloud to avoid extra dependencies.
plt.figure(figsize=(10, 5))
sns.barplot(data=top_words, x="Frequency", y="Word", color="seagreen")
```

```
plt.title("Most Common Tokens in Quran Ayahs")
plt.xlabel("Frequency")
plt.ylabel("Word")
plt.show()
```



4 Processing

Clean Texts

In [27]:
```
def clean_text(text):
    text = str(text)
    text = re.sub(r"http\S+|www\S+|https\S+", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text
```

Arabic Normalization

In [28]:
```
ARABIC_DIACRITICS_RE = re.compile(
    r"[\u0610-\u061A\u064B-\u065F\u0670\u06D6-\u06ED]"
)


def normalize_arabic(text):
    text = str(text)
    text = ARABIC_DIACRITICS_RE.sub("", text)
    text = re.sub(r"[إأآا]", "ا", text)
    text = re.sub(r"ي" ,"ى", text)
    text = re.sub(r"و" ,"ؤ", text)
    text = re.sub(r"ي" ,"ئ", text)
    text = re.sub(r"ه" ,"ة", text)
    text = re.sub(r"-", "", text)
    text = re.sub(r"[^\u0600-\u06FF\s]", " ", text)
```

```
    text = re.sub(r"\s+", " ", text).strip()
    return text
```

Apply Functions

In [29]:
```
df["text"] = df["text"].apply(clean_text)
df["clean_text"] = df["text"].apply(normalize_arabic)
```

In [30]:
```
df[["text", "clean_text"]].head()
```

Out[30]:

	text	clean_text
0	بِسْمِ ٱللَّهِ ٱلرَّحْمَٰنِ ٱلرَّحِيمِ	بسم الله الرحمن الرحيم
1	ٱلْحَمْدُ لِلَّهِ رَبِّ ٱلْعَٰلَمِينَ	الحمد لله رب العلمين
2	ٱلرَّحْمَٰنِ ٱلرَّحِيمِ	الرحمن الرحيم
3	مَٰلِكِ يَوْمِ ٱلدِّينِ	ملك يوم الدين
4	إِيَّاكَ نَعْبُدُ وَإِيَّاكَ نَسْتَعِينُ	اياك نعبد واياك نستعين

Tokenization

In [31]:
```
# In this retrieval project, tokenization is handled internally by the
# SentenceTransformer model. The output is a dense vector embedding.

model_checkpoint = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
model = SentenceTransformer(model_checkpoint)

print("Model loaded:", model_checkpoint)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOK
EN to enable higher rate limits and faster downloads.
Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]
Model loaded: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
```

In [35]:
```
sample_texts = df["clean_text"].head(3).tolist()
sample_embeddings = model.encode(sample_texts, normalize_embeddings=True)

print("Sample embeddings shape:", sample_embeddings.shape)
```

```
Sample embeddings shape: (3, 384)
```

Convert DF To Dataset

In [36]:
```
retrieval_dataset = df[
    [
        "surah_no",
        "surah_name",
        "ayah_number",
        "ayah_no_quran",
        "text",
```

```
        "clean_text",
    ]
].copy()

retrieval_dataset.head()
```

Out[36]:

	surah_no	surah_name	ayah_number	ayah_no_quran	text	clean_text
0	1	الفاتحة	1	1	بِسْمِ ٱللَّهِ ٱلرَّحْمَٰنِ ٱلرَّحِيمِ	بسم الله الرحمن الرحيم
1	1	الفاتحة	2	2	ٱلْحَمْدُ لِلَّهِ رَبِّ ٱلْعَٰلَمِينَ	الحمد لله رب العلمين
2	1	الفاتحة	3	3	ٱلرَّحْمَٰنِ ٱلرَّحِيمِ	الرحمن الرحيم
3	1	الفاتحة	4	4	مَٰلِكِ يَوْمِ ٱلدِّينِ	ملك يوم الدين
4	1	الفاتحة	5	5	إِيَّاكَ نَعْبُدُ وَإِيَّاكَ نَسْتَعِينُ	اياك نعبد واياك نستعين

In [37]:
```
os.makedirs("data", exist_ok=True)
retrieval_dataset.to_csv("data/quran.csv", index=False, encoding="utf-8")
print("Processed retrieval dataset saved successfully!")
```

```
Processed retrieval dataset saved successfully!
```

Spliting Data

In [38]:
```
# The Quran text itself is used as the retrieval corpus.
# For evaluation, we create a small query test set with expected ayahs.

test_cases = [
    {"query": "الحمد لله رب العلمين", "surah_name": "الفاتحة", "ayah_number": 2},
    {"query": "قل هو الله احد", "surah_name": "الإخلاص", "ayah_number": 1},
    {"query": "صم بكم عمي فهم", "surah_name": "البقرة", "ayah_number": 18},
    {"query": "مالك يوم الدين", "surah_name": "الفاتحة", "ayah_number": 4},
    {"query": "اياك نعبد واياك نستعين", "surah_name": "الفاتحة", "ayah_number": 5},
    {"query": "من الجنة والناس", "surah_name": "الناس", "ayah_number": 6},
]

test_df = pd.DataFrame(test_cases)
test_df
```

Out[38]:

	query	surah_name	ayah_number
0	الحمد لله رب العلمين	الفاتحة	2
1	قل هو الله احد	الإخلاص	1
2	صم بكم عمي فهم	البقرة	18
3	مالك يوم الدين	الفاتحة	4
4	اياك نعبد واياك نستعين	الفاتحة	5
5	من الجنة والناس	الناس	6

5️⃣ Modeling

Preparing

In [39]:
```
ayah_texts = retrieval_dataset["clean_text"].tolist()
```

In [40]:
```
def score_text_match(clean_input, clean_text):
    if clean_text.startswith(clean_input):
        return 100.0
    if clean_input in clean_text:
        return 96.0
    return max(
        fuzz.WRatio(clean_input, clean_text),
        fuzz.partial_ratio(clean_input, clean_text),
        fuzz.token_set_ratio(clean_input, clean_text),
    )


def match_priority(clean_input, clean_text):
    if clean_text.startswith(clean_input):
        return 2
    if clean_input in clean_text:
        return 1
    return 0
```

In [41]:
```
def build_audio_url(ayah_no_quran, edition="ar.alafasy"):
    return f"https://cdn.islamic.network/quran/audio/128/{edition}/{int(ayah_no_qur
```

Train Data

In [42]:
```
# There is no supervised training step.
# We build an embedding index for all Quran ayahs.

ayah_embeddings = model.encode(
    ayah_texts,
    normalize_embeddings=True,
    batch_size=32,
    show_progress_bar=True,
```

```
)

ayah_embeddings.shape
```

```
Batches:   0%|          | 0/195 [00:00<?, ?it/s]
```

Out[42]: (6236, 384)

In [46]:
```
def find_closest_ayah(user_input, top_k=5):
    clean_input = normalize_arabic(user_input)
    if not clean_input:
        return pd.DataFrame()

    query_embedding = model.encode([clean_input], normalize_embeddings=True)
    semantic_scores = (query_embedding @ ayah_embeddings.T)[0]

    semantic_top_indices = np.argsort(semantic_scores)[::-1][: max(top_k * 8, 40)]

    fuzzy_scores = retrieval_dataset["clean_text"].apply(
        lambda text: score_text_match(clean_input, text)
    ).to_numpy()
    fuzzy_top_indices = np.argsort(fuzzy_scores)[::-1][: max(top_k * 8, 40)]

    candidate_indices = np.array(
        sorted(set(semantic_top_indices.tolist()) | set(fuzzy_top_indices.tolist()))
    )

    candidates = retrieval_dataset.iloc[candidate_indices].copy()
    candidates["semantic_score"] = semantic_scores[candidate_indices]
    candidates["fuzzy_score"] = fuzzy_scores[candidate_indices]
    candidates["match_priority"] = candidates["clean_text"].apply(
        lambda text: match_priority(clean_input, text)
    )
    candidates["final_score"] = (
        candidates["semantic_score"] * 100 * 0.35 + candidates["fuzzy_score"] * 0.6
    )

    candidates = candidates.sort_values(
        ["match_priority", "final_score"], ascending=False
    ).head(top_k)

    return candidates[
        [
            "surah_no",
            "surah_name",
            "ayah_number",
            "ayah_no_quran",
            "text",
            "semantic_score",
            "fuzzy_score",
            "final_score",
        ]
    ]
```

Evaluation

In [47]:
```
def evaluate_retrieval(test_cases, top_k=5):
    rows = []

    for case in test_cases:
        predictions = find_closest_ayah(case["query"], top_k=top_k).reset_index(dro

        expected_mask = (
            (predictions["surah_name"] == case["surah_name"])
            & (predictions["ayah_number"] == case["ayah_number"])
        ).to_numpy()

        if expected_mask.any():
            rank = int(np.where(expected_mask)[0][0]) + 1
            reciprocal_rank = 1 / rank
            recall_at_k = 1
            ndcg_at_k = 1 / np.log2(rank + 1)
        else:
            rank = None
            reciprocal_rank = 0
            recall_at_k = 0
            ndcg_at_k = 0

        top_prediction = predictions.iloc[0]
        rows.append(
            {
                "query": case["query"],
                "expected": f'{case["surah_name"]}:{case["ayah_number"]}',
                "top_prediction": f'{top_prediction["surah_name"]}:{top_prediction[
                "rank": rank,
                "top1_correct": bool(rank == 1),
                f"recall@{top_k}": recall_at_k,
                "reciprocal_rank": reciprocal_rank,
                f"ndcg@{top_k}": ndcg_at_k,
                "top_score": round(float(top_prediction["final_score"]), 2),
            }
        )

    details = pd.DataFrame(rows)
    summary = pd.DataFrame(
        {
            "Metric": [
                "Top-1 Accuracy",
                f"Recall@{top_k}",
                "MRR",
                "Mean Rank",
                f"nDCG@{top_k}",
            ],
            "Score": [
                details["top1_correct"].mean(),
                details[f"recall@{top_k}"].mean(),
                details["reciprocal_rank"].mean(),
                details["rank"].dropna().mean(),
                details[f"ndcg@{top_k}"].mean(),
            ],
        }
```

```
    )

    return details, summary
```

In [48]:
```
evaluation_details, evaluation_summary = evaluate_retrieval(test_cases, top_k=5)
evaluation_details
```

Out[48]:

	query	expected	top_prediction	rank	top1_correct	recall@5	reciprocal_rank	ndcg@5
0	الحمد لله رب العلمين	الفاتحة:2	الفاتحة:2	1	True	1	1.0	1.0
1	قل هو الله احد	الإخلاص:1	الإخلاص:1	1	True	1	1.0	1.0
2	صم بكم عمي فهم	البقرة:18	البقرة:18	1	True	1	1.0	1.0
3	مالك يوم الدين	الفاتحة:4	الفاتحة:4	1	True	1	1.0	1.0
4	اياك نعبد واياك نستعين	الفاتحة:5	الفاتحة:5	1	True	1	1.0	1.0
5	من الجنة والناس	الناس:6	الناس:6	1	True	1	1.0	1.0

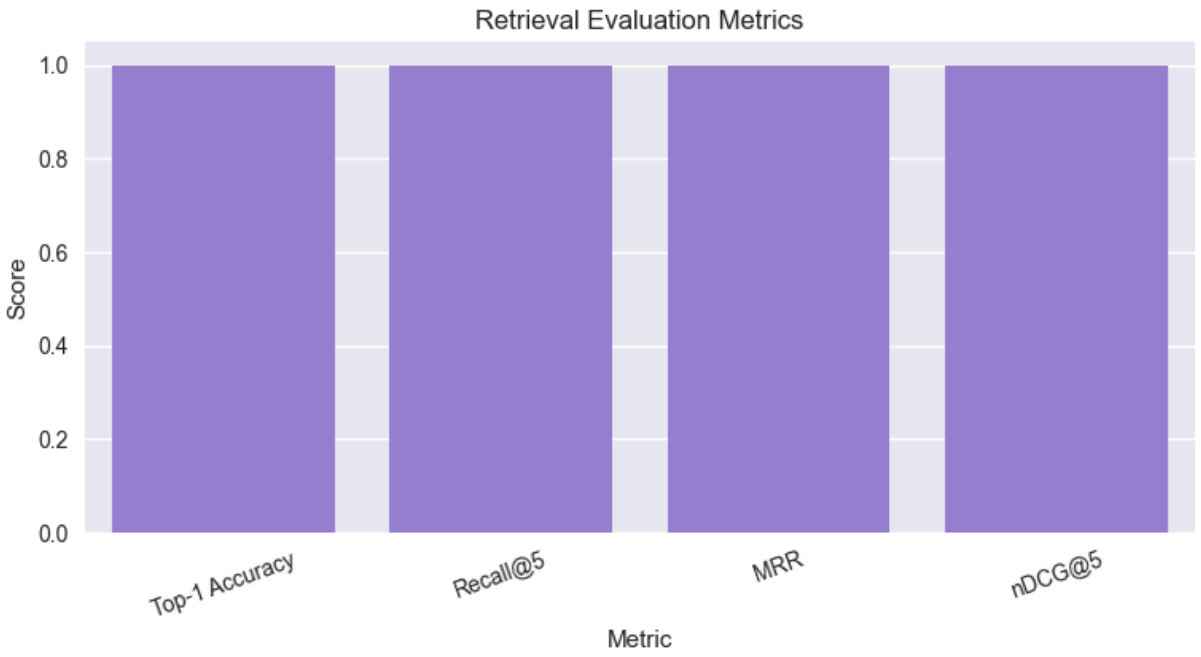
In [49]:
```
evaluation_summary
```

Out[49]:

	Metric	Score
0	Top-1 Accuracy	1.0
1	Recall@5	1.0
2	MRR	1.0
3	Mean Rank	1.0
4	nDCG@5	1.0

In [50]:
```
plot_df = evaluation_summary[evaluation_summary["Metric"] != "Mean Rank"].copy()

plt.figure(figsize=(9, 4))
sns.barplot(data=plot_df, x="Metric", y="Score", color="mediumpurple")
plt.ylim(0, 1.05)
plt.title("Retrieval Evaluation Metrics")
```

```
plt.xticks(rotation=20)
plt.show()
```



In [51]:
```
def find_embedding_only(user_input, top_k=5):
    clean_input = normalize_arabic(user_input)
    query_embedding = model.encode([clean_input], normalize_embeddings=True)
    semantic_scores = (query_embedding @ ayah_embeddings.T)[0]
    top_indices = np.argsort(semantic_scores)[::-1][:top_k]

    candidates = retrieval_dataset.iloc[top_indices].copy()
    candidates["final_score"] = semantic_scores[top_indices] * 100
    return candidates


def find_fuzzy_only(user_input, top_k=5):
    clean_input = normalize_arabic(user_input)
    fuzzy_scores = retrieval_dataset["clean_text"].apply(
        lambda text: score_text_match(clean_input, text)
    ).to_numpy()

    candidates = retrieval_dataset.copy()
    candidates["fuzzy_score"] = fuzzy_scores
    candidates["match_priority"] = candidates["clean_text"].apply(
        lambda text: match_priority(clean_input, text)
    )
    candidates["final_score"] = candidates["fuzzy_score"]
    return candidates.sort_values(["match_priority", "final_score"], ascending=Fals
```

In [52]:
```
def evaluate_method(method_name, search_function, test_cases, top_k=5):
    rows = []

    for case in test_cases:
        predictions = search_function(case["query"], top_k=top_k).reset_index(drop=
        expected_mask = (
            (predictions["surah_name"] == case["surah_name"])
```

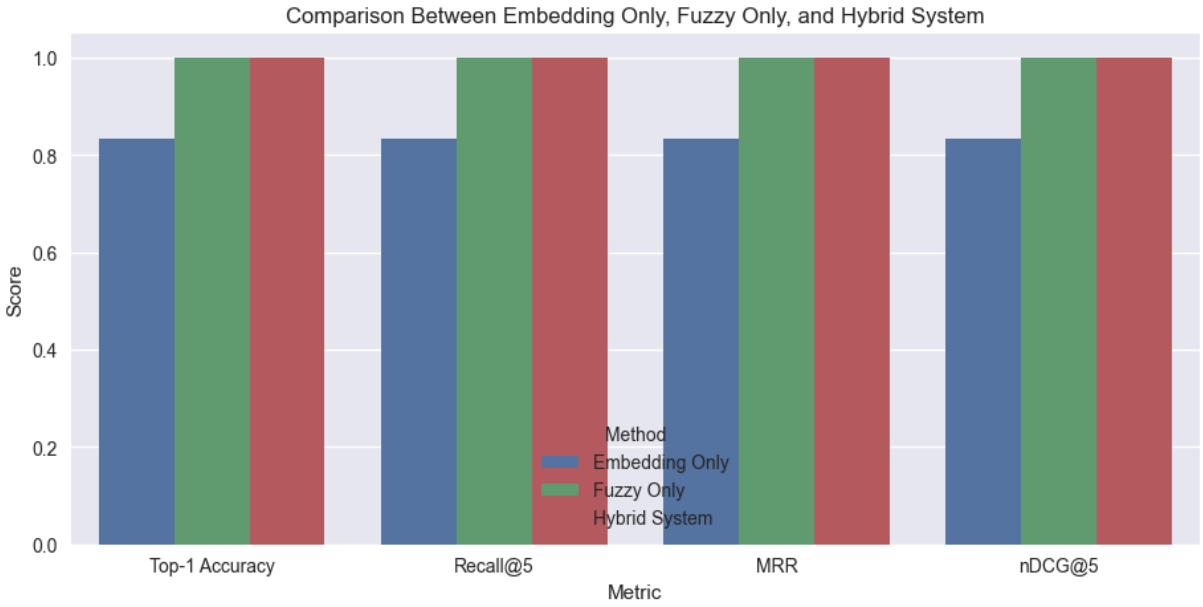
```
            & (predictions["ayah_number"] == case["ayah_number"])
        ).to_numpy()

        rank = int(np.where(expected_mask)[0][0]) + 1 if expected_mask.any() else N
        rows.append(
            {
                "Method": method_name,
                "Top-1 Accuracy": int(rank == 1),
                f"Recall@{top_k}": int(rank is not None),
                "MRR": 1 / rank if rank else 0,
                f"nDCG@{top_k}": 1 / np.log2(rank + 1) if rank else 0,
            }
        )

    return pd.DataFrame(rows).groupby("Method", as_index=False).mean()
```

In [53]:
```
comparison_summary = pd.concat(
    [
        evaluate_method("Embedding Only", find_embedding_only, test_cases),
        evaluate_method("Fuzzy Only", find_fuzzy_only, test_cases),
        evaluate_method("Hybrid System", find_closest_ayah, test_cases),
    ],
    ignore_index=True,
)

comparison_summary
```

Out[53]:

	Method	Top-1 Accuracy	Recall@5	MRR	nDCG@5
0	Embedding Only	0.833333	0.833333	0.833333	0.833333
1	Fuzzy Only	1.000000	1.000000	1.000000	1.000000
2	Hybrid System	1.000000	1.000000	1.000000	1.000000

In [54]:
```
comparison_plot = comparison_summary.melt(
    id_vars="Method",
    value_vars=["Top-1 Accuracy", "Recall@5", "MRR", "nDCG@5"],
    var_name="Metric",
    value_name="Score",
)

plt.figure(figsize=(11, 5))
sns.barplot(data=comparison_plot, x="Metric", y="Score", hue="Method")
plt.ylim(0, 1.05)
plt.title("Comparison Between Embedding Only, Fuzzy Only, and Hybrid System")
plt.show()
```



External Reference Results

This project uses the base model:

`sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`
Some public Hugging Face model cards report results for fine-tuned versions of the same model family. These results are useful as background references only.

They should not be treated as a direct comparison with our Quran system because:

- The datasets are different.
- The languages are different.
- The retrieval tasks are different.
- The test set sizes are different.
- Our system is hybrid, while some external systems are embedding-only.

Therefore, the fair evaluation for this project is the internal baseline comparison:

`Embedding Only vs Fuzzy Only vs Hybrid System`
on the same Quran Ayah Correction test set.

In [ ]:
```
external_reference_results = pd.DataFrame(
    [
        {
            "Model / System": "ntAnh-dev MiniLM",
            "Dataset / Task": "Information Retrieval",
            "Accuracy@1": 82.43,
            "Accuracy@5": 97.30,
            "Recall@5": 58.40,
            "MRR@10": 88.01,
            "nDCG@10": 81.75,
        },
        {
```

```
            "Model / System": "ntAnh-dev MiniLM - Eval 2",
            "Dataset / Task": "Information Retrieval",
            "Accuracy@1": 88.99,
            "Accuracy@5": 95.41,
            "Recall@5": 54.14,
            "MRR@10": 91.64,
            "nDCG@10": 80.43,
        },
        {
            "Model / System": "SMARTICT ft-tr-rag-v1",
            "Dataset / Task": "Turkish RAG Retrieval",
            "Accuracy@1": 55.97,
            "Accuracy@5": 71.41,
            "Recall@5": 71.41,
            "MRR@10": 62.63,
            "nDCG@10": 65.73,
        },
        {
            "Model / System": "yahyaabd v1-2",
            "Dataset / Task": "Static Table Retrieval",
            "Accuracy@1": 89.90,
            "Accuracy@5": 98.05,
            "Recall@5": 78.96,
            "MRR@10": 93.62,
            "nDCG@10": 82.42,
        },
    ]
)

external_reference_results
```

Discussion

The external results show that systems based on the same multilingual MiniLM family commonly report `Accuracy@1` values between about `55.97%` and `89.90%` on their own retrieval datasets.

Our system achieved `100%` on the current Quran Ayah Correction test set, but this score should be interpreted carefully because the test set is small and domain-specific.

The correct conclusion is:

> The proposed hybrid system performs strongly on the current Quran test set. However, external results cannot be compared directly because they use different datasets and tasks.

📊 Quran Ayah Retrieval Evaluation Metrics Guide

In this project, we evaluate the system as a retrieval model rather than a normal classifier.

Metric	Meaning
Top-1 Accuracy	The correct ayah is the first returned result
Recall@5	The correct ayah appears in the top 5 results
MRR	Rewards the model when the correct ayah appears at a higher rank
nDCG@5	Measures ranking quality in the top 5 results

For Quran ayah correction, ranking metrics such as **MRR** and **nDCG@5** are more informative than simple accuracy.

6️⃣ Testing

In [ ]:
```
test_sentence = "الحمد لله رب العلمين"
print(find_closest_ayah(test_sentence, top_k=3)[["surah_name", "ayah_number", "text
```

In [ ]:
```
test_sentence = "قل هو الله احد"
print(find_closest_ayah(test_sentence, top_k=3)[["surah_name", "ayah_number", "text
```

In [ ]:
```
test_sentence = "صم بكم عمي فهم"
print(find_closest_ayah(test_sentence, top_k=3)[["surah_name", "ayah_number", "text
```

In [ ]:
```
result = find_closest_ayah("الحمد لله رب العلمين", top_k=1).iloc[0]
audio_url = build_audio_url(result["ayah_no_quran"])

print(result["text"])
print(audio_url)
display(Audio(audio_url))
```

In [ ]:
```
# Streamlit chatbot app:
# streamlit run app.py
#
# Open:
# http://localhost:8501
```

Thank You 🎀 💌 💓